SQL Joins

1. Cartesian Product

Employees **(A)**

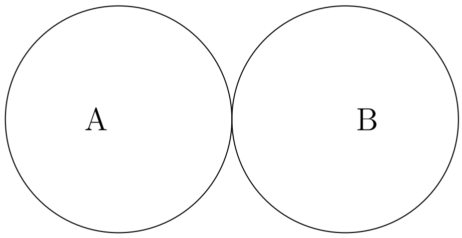
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments **(B)**

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Returns all possible combinations of rows from both tables. Size = $|A| \times |B|$.



SQL

```
SELECT *
FROM Employees CROSS JOIN Departments;
```

Output

emp_id	dept_id	dept_name
1	10	HR
1	20	IT
1	30	Sales
1	50	Finance
2	10	HR
2	20	IT
2	30	Sales
2	50	Finance
3	10	HR
3	20	IT
3	30	Sales
3	50	Finance
4	10	HR
4	20	IT
4	30	Sales
4	50	Finance

2. Inner Join

Employees **(A)**

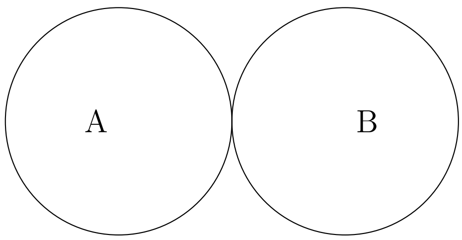
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments **(B)**

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Returns only rows with matching values in both tables.



SQL

```
SELECT e.emp_id, e.dept_id, d.dept_name
FROM Employees e
INNER JOIN Departments d
ON e.dept_id = d.dept_id;
```

Output

emp_id	dept_id	dept_name
1	10	HR
2	20	IT
3	30	Sales

3. Natural Join

Employees **(A)**

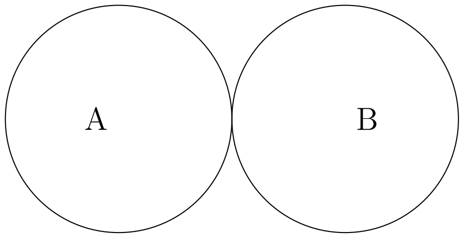
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments **(B)**

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Automatically joins tables on columns with the same name and removes duplicates in the output.



SQL

```
SELECT *
FROM Employees
NATURAL JOIN Departments;
```

Output

emp_id	dept_id	dept_name
1	10	HR
2	20	IT
3	30	Sales

4. Join with ON

Employees **(A)**

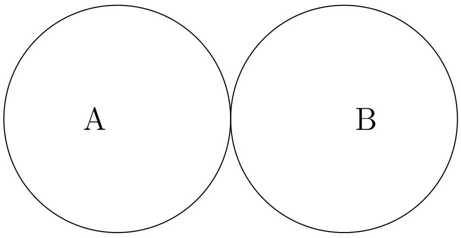
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments **(B)**

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Explicitly specifies the join condition. Works with INNER or OUTER joins.



SQL

```
SELECT e.emp_id, d.dept_name
FROM Employees e
JOIN Departments d
ON e.dept_id = d.dept_id;
```

Output

emp_id	dept_name
1	HR
2	IT
3	Sales

5. Join with USING

Employees **(A)**

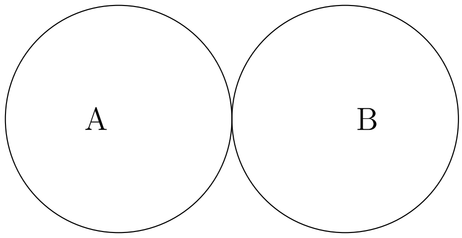
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments **(B)**

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Joins on a column with the same name in both tables. Removes one duplicate column in output.



SQL

```
SELECT emp_id, dept_id, dept_name
FROM Employees
JOIN Departments
USING (dept_id);
```

Output

emp_id	dept_id	dept_name
1	10	HR
2	20	IT
3	30	Sales

6. Left Outer Join

Employees **(A)**

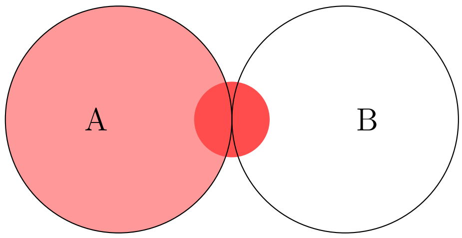
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments **(B)**

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Returns all rows from the left table and matching rows from the right. Unmatched right rows → NULL.



SQL

```
SELECT e.emp_id, e.dept_id, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_id = d.dept_id;
```

Output

emp_id	dept_id	dept_name
1	10	HR
2	20	IT
3	30	Sales
4	40	NULL

7. Right Outer Join

Employees **(A)**

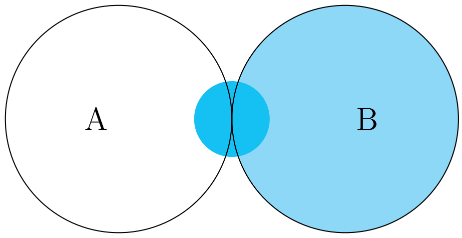
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments **(B)**

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Returns all rows from the right table and matching rows from the left. Unmatched left rows → NULL.



SQL

```
SELECT e.emp_id, e.dept_id, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_id = d.dept_id;
```

Output

emp_id	dept_id	dept_name
1	10	HR
2	20	IT
3	30	Sales
NULL	50	Finance

8. Full Outer Join

Employees (A)

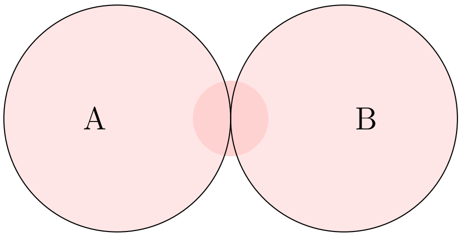
emp_id	dept_id
1	10
2	20
3	30
4	40

Departments (B)

dept_id	dept_name
10	HR
20	IT
30	Sales
50	Finance

Definition

Returns all rows from both tables. Unmatched sides → NULL.



SQL

```
SELECT e.emp_id, e.dept_id, d.dept_name
FROM Employees e
FULL OUTER JOIN Departments d
ON e.dept_id = d.dept_id;
```

Output

emp_id	dept_id	dept_name
1	10	HR
2	20	IT
3	30	Sales
4	40	NULL
NULL	50	Finance